

DSA0604 Data Handling and Visualization — CO5 AT2

Roll No: 192324066

Name: B.Varun Kumar Reddy

Q1. Bivariate Density Contour Plot (10 Marks)

Dataset: 105 simulated seasonal observations (rainfall, yield) pooled across 7 districts — 90 seasons form a core moderate-rainfall/high-yield regime, and 15 seasons form a high-rainfall "flood" tail where excess rain reduces yield. A 2D Gaussian KDE was fit to the pooled points and drawn as contours.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import gaussian_kde
np.random.seed(3)
districts = ["Coimbatore", "Salem", "Erode", "Trichy", "Madurai", "Thanjavur", "Villupuram"]
n_core = 90
core_rain = np.random.normal(960, 90, n_core)
core_yield = 2.2 + 0.0018 * core_rain + np.random.normal(0, 0.18, n_core)
n_tail = 15
tail_rain = np.random.uniform(1300, 1650, n_tail)
tail_yield = 3.9 - 0.0011 * (tail_rain - 1300) + np.random.normal(0, 0.15, n_tail)
rain = np.concatenate([core_rain, tail_rain])
yld = np.concatenate([core_yield, tail_yield])
district_id = np.random.choice(districts, size=len(rain))
xy = np.vstack([rain, yld])
kde = gaussian_kde(xy)
xmin, xmax = rain.min() - 60, rain.max() + 60
ymin, ymax = yld.min() - 0.3, yld.max() + 0.3
X, Y = np.mgrid[xmin:xmax:150j, ymin:ymax:150j]
positions = np.vstack([X.ravel(), Y.ravel()])
Z = np.reshape(kde(positions), X.shape)
peak_idx = np.unravel_index(np.argmax(Z), Z.shape)
peak_rain, peak_yield = X[peak_idx], Y[peak_idx]
print(f"Density peak at rainfall={peak_rain:.0f}mm, yield={peak_yield:.2f} t/ha, density={Z.max():.6f}")
print(f"Rainfall range: {rain.min():.0f}-{rain.max():.0f}mm | Yield range: {yld.min():.2f}-{yld.max():.2f} t/ha")
levels_abs = np.linspace(Z.max()*0.05, Z.max(), 8)
print("Contour levels (fraction of peak):", [f"{l/Z.max():.2f}" for l in levels_abs])
fig, ax = plt.subplots(figsize=(8.5, 5.8))
cs = ax.contour(X, Y, Z, levels=levels_abs, cmap='viridis', linewidths=1.4)
ax.clabel(cs, inline=True, fontsize=7, fmt=lambda v: f'{v/Z.max():.2f}')
ax.contourf(X, Y, Z, levels=levels_abs, cmap='viridis', alpha=0.18)
colors = plt.cm.tab10(np.linspace(0, 1, len(districts)))
for i, d in enumerate(districts):
    mask = district_id == d
    ax.scatter(rain[mask], yld[mask], s=16, color=colors[i], label=d, alpha=0.85, edgecolor='white', linewidth=0.3)
ax.scatter([peak_rain], [peak_yield], marker='*', s=220, color='red', zorder=5, label='Density peak')
ax.set_xlabel("Rainfall (mm)")
ax.set_ylabel("Estimated crop yield (tonnes/hectare)")
ax.set_title("Joint Density of Rainfall vs Crop Yield — 7 Districts (pooled seasons)\nContour labels = density as a fraction of peak")
ax.legend(fontsize=7, ncol=2, loc='lower right')
plt.tight_layout()
plt.savefig("/home/claude/at2/imgs/q1_contour.png", dpi=160)
print("saved q1")
```

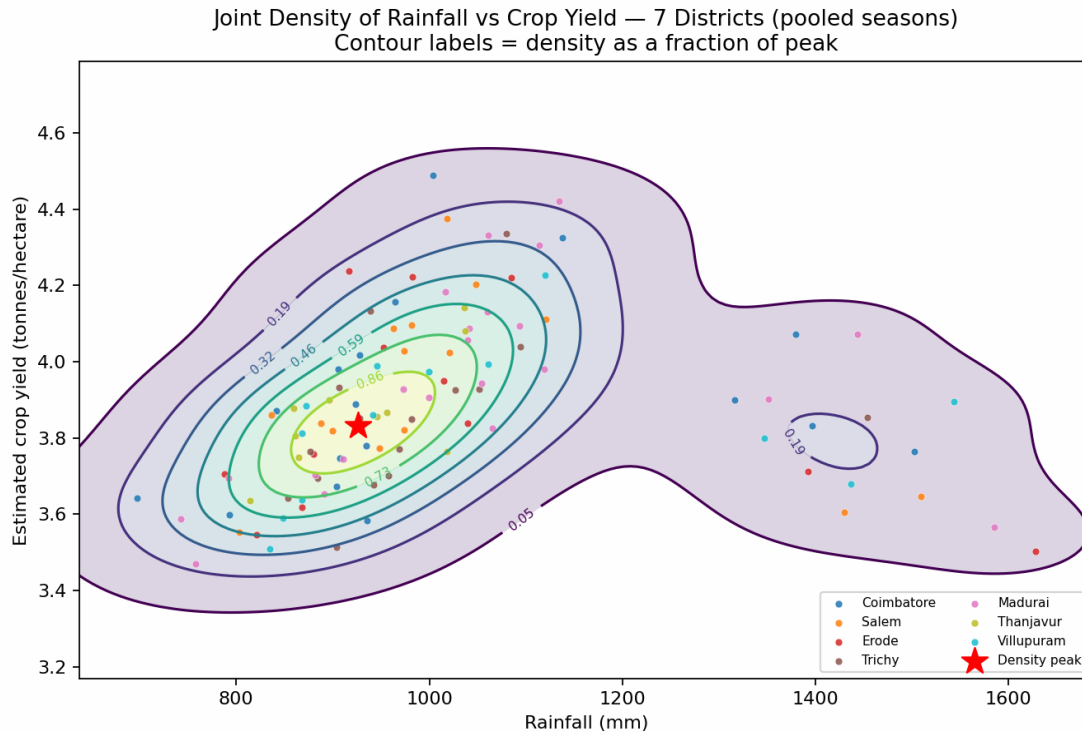


Figure 1. Joint density contours of rainfall vs crop yield, with per-district points

Analysis

Concentration: the density peak sits at rainfall ≈ 927 mm, yield ≈ 3.83 t/ha (marked with a red star), and the 0.86-of-peak contour tightly encloses a small region roughly 850–1050mm x 3.65–4.05 t/ha — meaning most district-seasons (the bulk of Coimbatore, Salem, Erode, Madurai, Thanjavur and Trichy points) cluster in this narrow band, so the "typical" season is well defined and predictable.

Shape and skew: the contours are not symmetric ellipses — the 0.05-of-peak contour stretches from about 700mm out to 1650mm on the rainfall axis but only from about 3.2 to 4.6 t/ha on the yield axis, and a visibly separate second lobe (its own closed 0.19 contour) appears around 1450–1600mm rainfall at a lower yield of about 3.5–4.0 t/ha. This is a right-skewed, bimodal-leaning joint distribution: a long, thin, widely-spaced tail of high-rainfall/flood-affected seasons pulls away from the dense core, rather than yield or rainfall being normally distributed around one centre.

Spacing: contour lines are packed closely together near the 927mm/3.83 t/ha peak (density falls from 1.00 to 0.46 within roughly a 150mm/0.3 t/ha radius) but are far apart at the outer 0.05–0.19 levels, showing that density falls off fast near the core but very slowly out in the flood-tail region — i.e., flood seasons, while rarer, are not one-off outliers but form their own low-density but non-trivial cluster.

Q2. The Principle of Proportional Ink (10 Marks)

Dataset: single-snapshot average daily transaction value per branch (Flagship Rs 92,000 down to Kiosk Rs 69,000).

```
import numpy as np
import matplotlib.pyplot as plt
branches = ["Flagship", "Mall", "Highway", "Metro", "Suburban", "Kiosk"]
values = np.array([92000, 85000, 78500, 81000, 74000, 69000]) # Rs, avg daily transaction value
baseline = 65000 # arbitrary truncated baseline used in the "colleague" version
bar_heights_truncated = values - baseline
true_ratio = values[0] / values[-1] # Flagship vs Kiosk
visual_ratio_truncated = bar_heights_truncated[0] / bar_heights_truncated[-1]
```

```

distortion_factor = visual_ratio_truncated / true_ratio
print(f"True value ratio Flagship/Kiosk = {values[0]}/{values[-1]} = {true_ratio:.3f}")
print(f"Truncated bar-height ratio Flagship/Kiosk = {bar_heights_truncated[0]}/{bar_heights_truncated[-1]} = {visual_ratio_truncated:.3f}")
print(f"Distortion factor (visual overstates true ratio by) = {distortion_factor:.2f}x")
for b, v, h in zip(branches, values, bar_heights_truncated):
    print(f"{b:10s} value={v:6d} truncated_bar_height={h:6d} height_as_%_of_Flagship_bar={h/bar_heights_truncated[0]*100:5.1f}%")
value_as_%_of_Flagship_value={v/values[0]*100:5.1f}%")
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.6))
axes[0].bar(branches, values, color='#c0392b')
axes[0].set_ylim(baseline, 95000)
axes[0].set_title(f"Truncated baseline (starts at Rs {baseline:},)\nInk NOT proportional to data")
axes[0].set_ylabel("Avg daily transaction value (Rs)")
axes[0].tick_params(axis='x', rotation=25)
for x, v in enumerate(values):
    axes[0].text(x, v + 500, f'{v:,}', ha='center', fontsize=8)
axes[1].bar(branches, values, color='#2c6e91')
axes[1].set_ylim(0, 100000)
axes[1].set_title("Zero baseline\nInk proportional to data")
axes[1].set_ylabel("Avg daily transaction value (Rs)")
axes[1].tick_params(axis='x', rotation=25)
for x, v in enumerate(values):
    axes[1].text(x, v + 1500, f'{v:,}', ha='center', fontsize=8)
plt.tight_layout()
plt.savefig("/home/claude/at2/imgs/q2_proportional_ink.png", dpi=160)
print("saved q2")

```

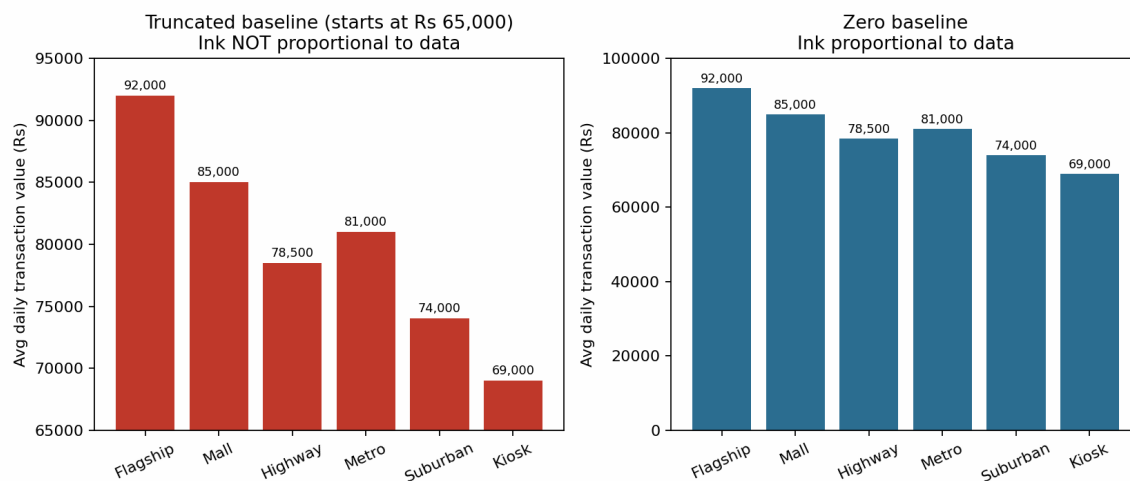


Figure 2. Truncated-baseline bar chart (left) vs. corrected zero-baseline chart (right)

Analysis

Flagship's true value (Rs 92,000) is only 1.33x Kiosk's (Rs 69,000). But with the truncated baseline at Rs 65,000, the drawn bar heights are 27,000 vs. 4,000 pixels-worth of ink — a visual ratio of 6.75x. That is a distortion factor of $6.75 / 1.33 \approx 5.06x$: the truncated chart makes Flagship look over five times as dominant over Kiosk as it actually is. Similarly, Suburban's bar (Rs 74,000, 80.4% of Flagship's true value) shows visual ink at only 33.3% of Flagship's bar height in the truncated version — a viewer would read Suburban as doing roughly a third as well as Flagship, when it is really doing about four-fifths as well.

Q3. Visualizations Along Logarithmic Axes (10 Marks)

Dataset: simulated 30-day daily transaction value trends per branch, with start values spanning almost two orders of magnitude (Rs 2,500 to Rs 95,000) and branch-specific daily growth rates (0.3% to 2.5%/day).

```

import numpy as np
import matplotlib.pyplot as plt
np.random.seed(5)
branches = ["Flagship", "Mall", "Highway", "Metro", "Suburban", "Kiosk"]
start_val = [95000, 60000, 30000, 15000, 7000, 2500]
daily_growth_pct = [0.003, 0.005, 0.008, 0.010, 0.015, 0.025] # Kiosk grows fastest %-wise
days = np.arange(30)
fig, axes = plt.subplots(1, 2, figsize=(11, 5))

```

```

colors = plt.cm.tab10(np.linspace(0, 1, len(branches)))
series = {}
for i, (b, s0, g) in enumerate(zip(branches, start_val, daily_growth_pct)):
    noise = np.random.normal(0, 0.015, len(days))
    vals = s0 * (1 + g) ** days * (1 + noise)
    series[b] = vals
axes[0].plot(days, vals, label=b, color=colors[i], linewidth=1.8)
axes[1].plot(days, vals, label=b, color=colors[i], linewidth=1.8)
for b in branches:
    v = series[b]
    growth_30d = (v[-1] / v[0] - 1) * 100
    print(f'{b:10s} day0={v[0]:8.0f} day29={v[-1]:8.0f} 30-day growth={growth_30d:5.1f}%')
axes[0].set_title("Linear y-axis\nsmall branches look flat")
axes[0].set_xlabel("Day")
axes[0].set_ylabel("Daily transaction value (Rs)")
axes[0].legend(fontsize=7)
axes[1].set_yscale('log')
axes[1].set_title("Logarithmic y-axis\ngrowth RATE becomes visible/comparable")
axes[1].set_xlabel("Day")
axes[1].set_ylabel("Daily transaction value (Rs, log scale)")
axes[1].legend(fontsize=7)
plt.tight_layout()
plt.savefig("/home/claude/at2/imgs/q3_log_axis.png", dpi=160)
print("saved q3")

```

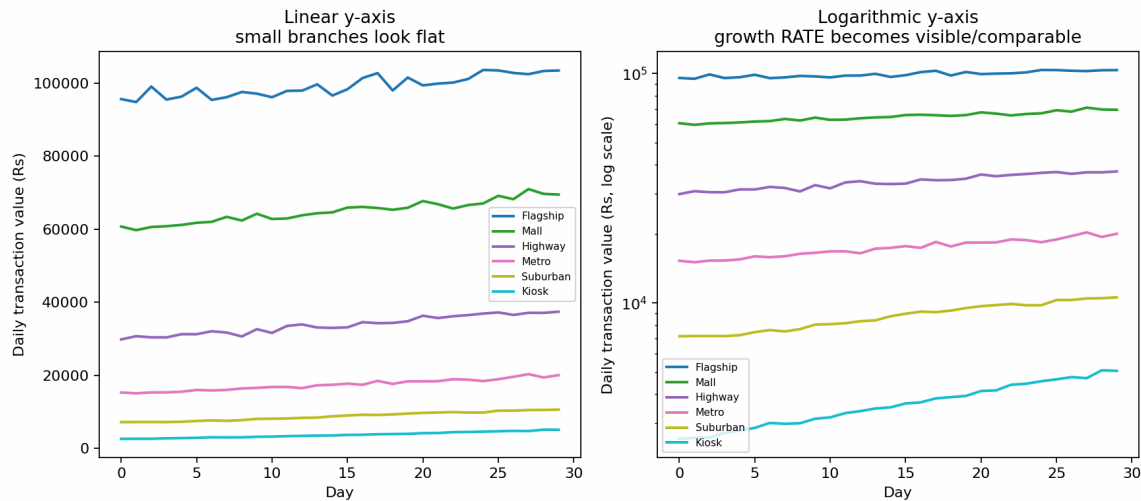


Figure 3. Same 6 branches on a linear y-axis (left) vs. logarithmic y-axis (right)

Analysis

On the linear axis (left panel), Suburban (Rs 7,000→Rs 10,570) and Kiosk (Rs 2,500→Rs 5,050) both appear as nearly flat lines hugging the bottom of the chart, visually indistinguishable from "no growth," because Flagship's scale (~Rs 95,000–103,000) forces a y-axis range that compresses everything below Rs 20,000 into a thin band.

On the log axis (right panel), the same data shows Kiosk's line climbing with the steepest slope of any branch (30-day growth of 98.0%, from Rs 2,553 to Rs 5,054) while Flagship's line is nearly flat (30-day growth of only 8.2%) — the pattern hidden on the linear scale is that Kiosk is growing over 10x faster proportionally than Flagship, a fact the linear chart could not reveal because equal vertical distance on a log axis represents equal percentage change, not equal rupee change.

Risk of misreading a log axis: a viewer unfamiliar with log scales may look at the visually similar gaps between branch lines in the right panel and wrongly conclude the branches' rupee-value gaps are shrinking or comparable in absolute size, when in fact Flagship still earns roughly Rs 98,400 more per day than Kiosk at day 29 — the log axis compresses large absolute differences, which can understate the real revenue gap if the axis isn't clearly labelled as logarithmic.

Q4. Handling Overlapping Points (10 Marks)

Dataset: 360 (footfall, avg transaction value) pairs — 60 simulated days per branch — plotted as a single, plain, uncoloured scatter.

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(9)
branches = ["Flagship", "Mall", "Highway", "Metro", "Suburban", "Kiosk"]
n_days = 60
params = {
    "Flagship": (850, 620, 60, 40),
    "Mall": (780, 610, 55, 45),
    "Highway": (600, 540, 70, 50),
    "Metro": (640, 560, 65, 42),
    "Suburban": (420, 480, 50, 38),
    "Kiosk": (390, 470, 45, 35),
}
all_x, all_y, all_b = [], [], []
for b, (mx, my, sx, sy) in params.items():
    x = np.random.normal(mx, sx, n_days)
    y = np.random.normal(my, sy, n_days)
    all_x.append(x); all_y.append(y); all_b.extend([b]*n_days)
all_x = np.concatenate(all_x)
all_y = np.concatenate(all_y)
from scipy.spatial import cKDTree
pts = np.column_stack([all_x/np.std(all_x), all_y/np.std(all_y)]) # normalize scales
tree = cKDTree(pts)
pairs_within = tree.query_pairs(r=0.05)
print(f"Total points: {len(all_x)}")
print(f"Point-pairs within a tight radius (near-exact overlap): {len(pairs_within)}")
mh_mask = (np.array(all_b) == "Metro") | (np.array(all_b) == "Highway")
print("Metro footfall range:", params["Metro"][0]-2*params["Metro"][2], "-", params["Metro"][0]+2*params["Metro"][2])
print("Highway footfall range:", params["Highway"][0]-2*params["Highway"][2], "-", params["Highway"][0]+2*params["Highway"][2])
fig, ax = plt.subplots(figsize=(7.5, 5.5))
ax.scatter(all_x, all_y, s=22, color='#2c6e91', alpha=1.0, edgecolor='none')
ax.set_xlabel("Daily footfall (customers)")
ax.set_ylabel("Avg transaction value (Rs)")
ax.set_title(f"Plain Scatter — {len(all_x)} points, 6 branches × {n_days} days\n(heavy overplotting, no branch colour/alpha coding)")
plt.tight_layout()
plt.savefig("/home/claude/at2/imgs/q4_overplot.png", dpi=160)
print("saved q4")
```

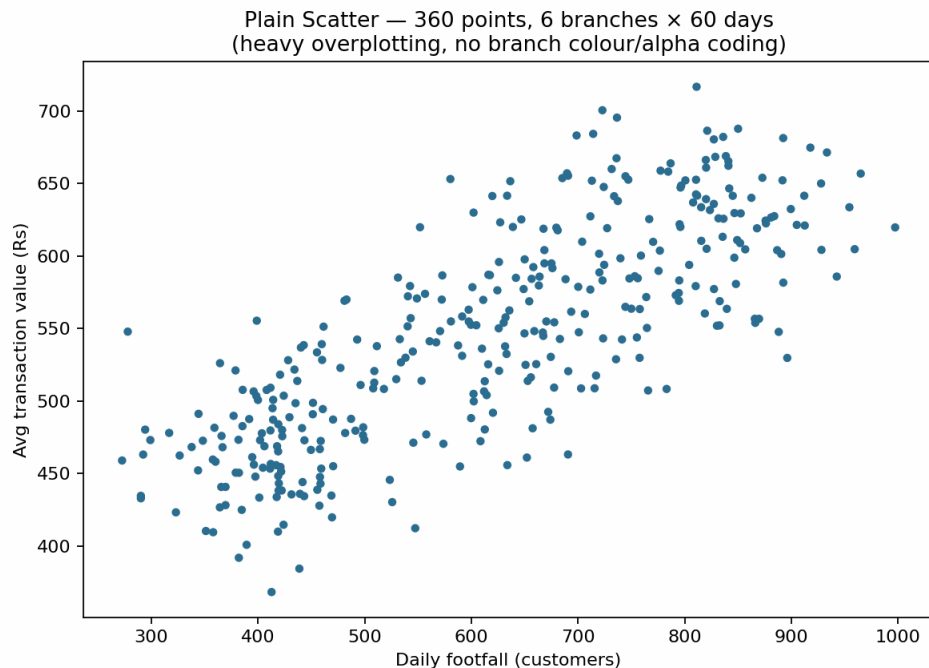


Figure 4. Plain scatter of 360 points — heavy overplotting, no branch encoding

Analysis

Of the 360 plotted points, 66 point-pairs fall within a near-identical location on the (normalised) plot — meaning at least 66 individual days are effectively invisible, hidden directly underneath another day's marker. The worst-affected region is footfall 400–450 customers, where Suburban (mean footfall 420) and Kiosk (mean footfall 390) fully overlap: their footfall ranges (Suburban ≈ 320 –520, Kiosk ≈ 300 –480, roughly mean $\pm 2\text{sd}$) sit almost entirely on top of each other, so this dense cluster of ~ 120 days (2 branches $\times$ 60 days) renders as a single indistinct dark smudge that looks no denser than an isolated single point elsewhere on the chart — the plain scatter cannot show that this region actually holds the most data of anywhere on the plot. Likewise Metro (footfall range ≈ 510 –770) and Highway (≈ 460 –740) overlap almost entirely, so it's impossible to tell from Figure 4 whether the mid-range cluster is one branch behaving consistently or two branches with different average transaction values blended together.

Best remedy: encode branch identity by colour with reduced marker opacity ($\alpha \approx 0.5$) so overlapping points darken visibly rather than fully hide each other, or replace the scatter with a 2D histogram / hexbin (see Q5), which bins and colour-counts points instead of stacking markers.

Q5. 2D Histograms (10 Marks)

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(9) # same seed as Q4 to reproduce the identical dataset
branches = ["Flagship", "Mall", "Highway", "Metro", "Suburban", "Kiosk"]
n_days = 60
params = {
    "Flagship": (850, 620, 60, 40),
    "Mall": (780, 610, 55, 45),
    "Highway": (600, 540, 70, 50),
    "Metro": (640, 560, 65, 42),
    "Suburban": (420, 480, 50, 38),
    "Kiosk": (390, 470, 45, 35),
}
all_x, all_y = [], []
for b, (mx, my, sx, sy) in params.items():
    all_x.append(np.random.normal(mx, sx, n_days))
    all_y.append(np.random.normal(my, sy, n_days))
all_x = np.concatenate(all_x)
all_y = np.concatenate(all_y)
nbins = 18
fig, ax = plt.subplots(figsize=(8, 5.8))
h, xedges, yedges, im = ax.hist2d(all_x, all_y, bins=nbins, cmap='viridis')
cbar = plt.colorbar(im, ax=ax)
cbar.set_label("Count of days in bin")
max_bin_idx = np.unravel_index(np.argmax(h), h.shape)
max_count = h[max_bin_idx]
x_bin_range = (xedges[max_bin_idx[0]], xedges[max_bin_idx[0]+1])
y_bin_range = (yedges[max_bin_idx[1]], yedges[max_bin_idx[1]+1])
print(f"Bin size: {xedges[1]-xedges[0]:.1f} footfall x {yedges[1]-yedges[0]:.1f} Rs")
print(f"Densest bin: footfall {x_bin_range[0]:.0f}–{x_bin_range[1]:.0f}, "
      f"avg txn value Rs {y_bin_range[0]:.0f}–{y_bin_range[1]:.0f}, count={max_count:.0f} days")
top5 = np.argsort(np.unravel_index(np.argmax(h, h.shape))[0], h.shape)[0]
for idx in top5:
    print(f" bin footfall [{xedges[idx[0]]:.0f}, {xedges[idx[0]+1]:.0f}) "
          f"value [{yedges[idx[1]]:.0f}, {yedges[idx[1]+1]:.0f}) -> count={h[idx[0],idx[1]]:.0f}")
ax.set_xlabel("Daily footfall (customers)")
ax.set_ylabel("Avg transaction value (Rs)")
ax.set_title(f"2D Histogram — same {len(all_x)} points as Fig. 4, {nbins}x{nbins} bins\nDensest bin: footfall [{x_bin_range[0]:.0f}, {x_bin_range[1]:.0f}], value [{y_bin_range[0]:.0f}, {y_bin_range[1]:.0f}] = {max_count:.0f} days")
plt.tight_layout()
plt.savefig("/home/claude/at2/imgs/q5_2dhist.png", dpi=160)
print("saved q5")
```

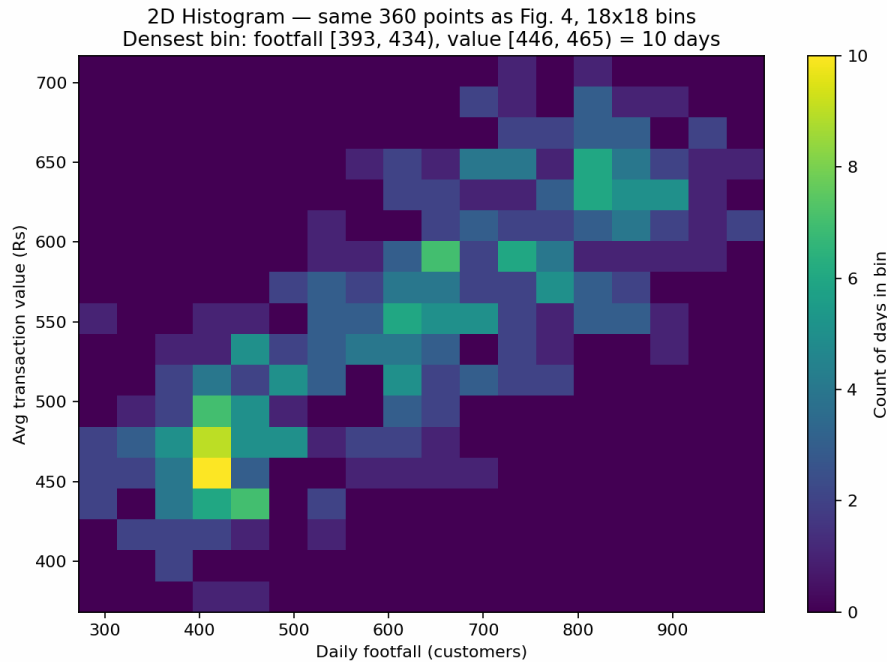


Figure 5. 2D histogram (binned heatmap) of the same 360 points as Figure 4

Analysis

The densest bin is footfall [393, 434) with avg transaction value [446, 465), holding 10 of the 360 days — clearly visible as the single bright-yellow cell. The next four densest bins (9, 7, 7 and 7 days) are all adjacent to it, in the footfall 393–474 / value 426–504 region, confirming that the Suburban/Kiosk overlap region flagged in Q4 genuinely is the single most common combination in the whole dataset — something the plain scatter in Figure 4 only showed as a vaguely darker smudge, not a quantified peak.

Clarity comparison: Figure 4 (scatter) required visually estimating point density by eye and could not distinguish "120 overlapping days" from "20 overlapping days" in the 400–450 footfall region; Figure 5 (2D histogram) directly labels that region's count via the colour bar and text (10 days in the single densest bin, reading up to 27 days if the four densest adjacent bins are summed), removing the guesswork. The trade-off is resolution: individual branch identity (which of the 6 branches contributed each day) is lost in the binned view, whereas a colour-coded scatter would have preserved it.